

Costing, Part A — Unit economics and the price basis

SIH 2026 · PS 26090 · What it costs to run this for a state, and then for the country

0. How to read every number in this document

Three markers, used on every figure. **Nothing is asserted without one.**

Marker	Meaning
[measured] MEASURED	Measured in this repository, on our hardware, with the file that produced it named
[list price] LIST	A vendor's published price at the time of writing. Must be re-verified before any procurement conversation. Currency converted at ₹88/USD
[derived] DERIVED	Arithmetic from the two above, with the assumption stated inline

The rule this follows is the same one that governs research/pricing/:

Never invent a number. A plausible-looking guess is worse than an empty cell — an empty cell is honest about what we do not know; a guess is not.

Every ₹ below is re-derivable. If a vendor price moves 30%, the *conclusions* do not change, because the conclusions rest on ratios, not on absolute prices.

1. The measured basis — everything else is built on these seven numbers

[measured] All from research/segmentation/RESULTS.md and docs/app/Future-Implementations.md §4, measured 2026-08-28 and 2026-09-02.

#	Fact	Value	Source
1	BiRefNet inference, warm	402 ms (RTX 4060) · 645 ms (RTX 2050)	Segmentation benchmark
2	BiRefNet VRAM	1615 MiB	Same
3	BiRefNet-lite inference /	273 ms / 851 MiB	Same — held in

#	Fact	Value	Source
	VRAM		reserve
4	CPU stages per photo	901 ms (decode 100 + gate 375 + master 168 + tier 124 + crop 54 + encode×3 80)	Latency profile
5	Total wall-clock compute	≈1.3 s	Same
6	Cold model load	3486 ms , first inference 899 ms	Same
7	Object-store round trip, Sydney	0.81–1.13 s × 8 serialized ≈ 7 s	Same

The most important structural fact hidden in that table: of the 1.3 s, **only 402 ms is GPU**. The other 901 ms is CPU work that does not touch the card.

[derived] **Consequence — a 3.2× capacity gain before buying anything.** Today one worker thread does CPU and GPU work serially, so throughput is $1 / 1.3 \text{ s} = \mathbf{0.77 \text{ photos/s}}$. Separate them — a CPU pool feeding a serialized GPU queue — and throughput becomes GPU-bound at $1 / 0.402 \text{ s} = \mathbf{2.49 \text{ photos/s}}$. Same hardware, same model, **3.2× the photographs**. This is the cheapest capacity in the entire plan and it is a refactor, not a purchase.

[derived] **And a second lever on top of it:** BiRefNet-lite at 273 ms gives **3.66 photos/s** — a further **1.47×** — at **half the VRAM**, so two lite workers fit on a card that held one full worker. Already benchmarked. Combined ceiling: **4.8× today's throughput with zero capital spend.**

2. Workload model — the assumptions, stated so they can be attacked

[derived] Everything downstream scales off these. They are deliberately **pessimistic** (higher usage = higher cost), so real costs should come in under, not over.

Parameter	Planning value	Reasoning
Products listed per active artisan per month	3	Steady state after the initial catalog burst
Photographs per product	3	One main, two secondary
Photographs per artisan per month	10	Rounded up from 9

Parameter	Planning value	Reasoning
Initial catalog burst, per artisan, one-time	20 products / 60 photos	Digitising existing stock
Voice interpretation calls per product	2	≤5 possible; the on-device tier usually wins, so most send nothing
Description generations per product	1	EN + HI in one call
Active-artisan ratio of those onboarded	40%	Conservative for a government scheme
Effective peak hours per day	8	Daylight, rural
Design headroom on peak	50%	Never plan to 100%

[derived] **Capacity per GPU, from #1 and the pipelined figure:**

2.49 photos/s × 3,600 × 8 effective hours	= 71,700 photos/day at full rate
× 50% headroom	= 35,850 photos/day
× 30 days	≈ 1,075,000 photos/month per GPU
÷ 10 photos per artisan per month	≈ 107,000 artisans per GPU

[derived] One GPU serves roughly 100,000 artisans. With BiRefNet-lite, roughly 158,000.

The GPU is not what limits this system, and that is the single most important costing fact in this document. Every intuition that "AI is expensive" assumes per-image API pricing. We self-host the model that runs on every photograph, precisely so that this line stays flat.

3. Marginal cost per product — where the money actually goes

[derived] Per **product** (3 photographs), at the assumptions in §2.

3.1 GPU compute

[list price] Cloud GPU, ap-south-1, list on-demand — **verify before quoting:**

Instance	GPU	~\$/hr	~₹/hr
g4dn.xlarge	T4 16 GB	0.526	₹46
g6.xlarge	L4 24 GB	~0.80	₹70

[derived] At the pipelined 2.49 photos/s $\Rightarrow$ 8,964 photos/hour $\Rightarrow$ **₹0.0051 per photograph** on a T4 at ₹46/hr. **Three photographs $\Rightarrow$ ₹0.015 per product.**

Even at today's un-pipelined 0.77 photos/s it is ₹0.017/photo — **₹0.05 per product.**

GPU inference is not a cost line. It is a rounding error. This is the direct result of decision "CV models self-hosted, LLM via API" (docs/decisions.md) — "per-image API pricing kills unit economics on the background-removal path, which runs on every photo."

[list price] **The counterfactual, for contrast:** a hosted background-removal API at ~\$0.02/image $\approx$ **₹1.76 per photograph** — **345 $\times$ our marginal cost.** At 10 million photographs a month that is **₹17.6 crore a year** against roughly **₹5 lakh** of self-hosted compute. That single decision is the largest cost avoidance in the project.

3.2 LLM API

[list price] List prices, per million tokens — **verify:**

Model	Role	In	Out
deepseek/deepseek-v4-flash (OpenRouter)	Voice $\rightarrow$ field value; slot harvest	~\$0.10	~\$0.30
claude-sonnet-5	EN + HI description, category mapping	~\$3.00	~\$15.00
(vision model, TBD)	/catalog/prefill — photo $\rightarrow$ nullable field guesses	—	—

[derived] Per product:

Call	Tokens (in / out)	Cost
Voice interpret $\times$ 2	400 / 60 each	\$0.00012 $\approx$ ₹0.01
Description, EN + HI + structured fields $\times$ 1	1,800 / 900	\$0.0189 $\approx$ ₹1.66
Subtotal		$\approx$ ₹1.67 per product

[warning] Two changes from the 2026-09-03 merge. The comparables-normalisation call is gone — comps.normalize() was deleted, so no model touches the price path at all now. And /catalog/prefill is built, which adds a vision call this table does not yet price: vision tokens cost more than text, and this is the one line in the cost model that will grow. It also buys back interview time, so net cost per product may fall — but that must be measured, not assumed.

The description call is 99% of our LLM spend. Everything else is noise. That is the line to optimise, and §5 of Part B gives the crossover volume at which self-hosting it becomes cheaper.

3.3 Speech — the single largest variable, and it is a procurement question not a technical one

[warning] Updated 2026-09-03. Speech is now live on Sarvam (bulbul:v3 TTS, Sarvam ASR, web/api/routers/voice.py). Bhashini is the strategic target, not the current provider. The cost model below therefore has a real middle row, not two hypotheticals.

Path	Per product (≈72 s speech in, plus TTS out)	Status
Bhashini	₹0 on the free prototyping tier	[open] docs/decisions.md #5 open — unused, terms unconfirmed
Sarvam — what runs today	[list price] rate not yet confirmed for volume — verify	[yes] Live and measured working
[list price] Global commercial STT	~\$0.006–0.016 per 15 s ⇒ ~\$0.03–0.08 ≈ ₹2.60–₹7.00	Ceiling, not our path

[derived] At 10 million products a year, the spread between the free tier and the commercial ceiling is ₹0 versus ₹26–70 crore. Sarvam sits somewhere between and we do not yet know where — which makes its volume rate the single most valuable unknown in this document.

Two things to establish, and neither is engineering work: Sarvam's committed-volume pricing, and whether Bhashini's terms cover us. The provider swap costs two functions — sarvam_tts and sarvam_asr are the only code that changes — so this is a procurement decision with an already-built escape hatch, which is the cheapest position to negotiate from.

And TTS output is already off the variable line entirely. Web Speech API for Indic TTS is on the rejected list in CLAUDE.md (unreliable voice availability across handsets) — pre-generated audio instead. [derived] Roughly 2,000 strings × 5 languages, a one-time ₹10,000–20,000, then ₹0 marginal forever, with no rate limit and no latency. A decision taken for reliability removes a recurring cost line.

3.4 Storage and bandwidth

[derived] Per product, with rule 2 (never destroy the original; store a recipe, render on demand):

Object	Size
Originals ×3, private bucket	~12 MB
2000 px masters ×3	~1.8 MB
Rendered variants (primary only; rest rendered on demand from the recipe)	~0.4 MB
Total	≈14 MB per product

Object	Size
[list price] Object storage, per GB-month: AWS S3 ap-south-1 Standard ~\$0.025 ≈ ₹2.20 · Cloudflare R2 ~\$0.015 ≈ ₹1.32 · S3 Glacier Instant ~\$0.004 ≈ ₹0.35.	

[derived] ₹0.031/product/month on S3, ₹0.019 on R2. Tiering the originals to archive after 90 days cuts the total by ~70%, because originals are 86% of the bytes and are read almost never — they exist for rule 2, not for serving.

[list price] Egress is where the real money is, and it is a vendor choice, not an engineering one:

Provider	Egress per GB	[derived] At 5 TB/month	[derived] At 100 TB/month
AWS ap-south-1	~\$0.1093 ≈ ₹9.62	₹49,200/mo	₹9.84 lakh/mo
Cloudflare R2	\$0 — zero egress	₹0	₹0

[derived] Choosing R2 over S3 for the public bucket saves ~₹1.2 crore a year at national scale, for an afternoon of work. The marketplace is a public catalog of images; egress is the product. This is the second-largest cost decision after Bhashini, and neither of them is a hard problem.

3.5 The per-product total

[derived] Assembling §3.1–§3.4:

Line	With Bhashini	With commercial ASR
GPU compute (3 photos)	₹0.02	₹0.02
LLM — description	₹1.66	₹1.66
LLM — interpret + comps	₹0.02	₹0.02
Speech (ASR) — Sarvam, rate unconfirmed	₹0.00	₹4.50
Storage (year 1, R2, tiered)	₹0.10	₹0.10
Egress (R2)	₹0.00	₹0.00
Marginal cost per product	≈ ₹1.80	≈ ₹6.30

[derived] Per artisan per year, at 36 products (3/month):

Marginal cost ₹65/artisan/year ₹227/artisan/year

The comparison that makes this land — [list price] market rates, verify before quoting:

Alternative	Typical cost
Manual product cataloging service	₹200–500 per SKU
GeM catalogue consultant, per seller onboarding	₹5,000–15,000 one-time
Ours, per artisan, per year, unlimited listings	₹65

A GeM consultant charges more to onboard **one** seller than we spend on **a hundred artisans for a year**. That is the economic argument, and it does not depend on any number being precise to better than a factor of two.

4. What is deliberately NOT in the marginal cost, and must not be forgotten

Marginal cost is the seductive number. These four lines are not marginal, do not shrink with scale, and at national scale **three of them are larger than everything in §3**.

Line	Why it is not marginal	Where it is costed
GeM order reconciliation labour	GeM has no order API. A human reads the dashboard	Part B §4 — the largest single line at scale
TCS collection + monthly GSTR-8	We are a seller-side ECO (CBIC Circular 194/06/2023). Compliance is ours	Part B §4
DPDP obligations	Named DPO, consent manager, breach process, retention	Part B §4
Fixed infrastructure	Postgres HA, Redis, app tier, CDN, monitoring — these exist at 2,000 artisans and at 10 lakh	Part B §2

A costing that only shows marginal cost is a costing designed to win an argument, not to run a programme. Part B carries the full picture.

5. The three sentences to say if a judge asks about cost

1. "One GPU serves about a hundred thousand artisans, because we self-host the model that runs on every photograph instead of paying per image — that one decision avoids about ₹17 crore a year at national scale."
2. "Marginal cost is about ₹65 per artisan per year. A GeM consultant charges ₹5,000 to onboard one seller."
3. "Our largest cost is not compute. It is a human reading the GeM dashboard, because GeM has no order API — and a single MoU would remove it."